

CP468 Artificial Intelligence

Assignment 2: Connect Four AI

Weight: 10%

Group Assignment

Submit: A single A2.zip file (see Deliverables)

Overview

In this assignment, your team will develop intelligent agents to play Connect Four, a classic two-player adversarial game. You will implement three agents of increasing intelligence and compare their performance through systematic experiments.

Rather than building a sophisticated game application, this assignment focuses on applying the AI techniques covered in this course — adversarial search and heuristic evaluation. A polished interface is not the point; correct algorithms and sound experiments are.

Connect Four Game

Connect Four is played on a 7×6 board (7 columns, 6 rows). Two players alternately drop a disc into one of the seven columns; a disc falls to the lowest empty cell of that column.

The first player to connect four consecutive discs horizontally, vertically, or diagonally wins. If the board fills with no winner, the game is a draw.

A text-based interface is sufficient. A graphical interface is optional.

Assignment Requirements

Requirement 1: Game Engine [1.5 marks]

Implement a functional Connect Four engine that supports:

- Board representation
- Legal-move generation
- Move execution (with correct “gravity”)
- Win detection (horizontal, vertical, and both diagonals)
- Draw detection (full board, no winner)

Your engine must expose a clean interface that the agents call (e.g. `legal_moves`, `apply_move`, `is_terminal`, `winner`), so all three agents share the same rules.

Requirement 2: AI Agents

Implement the following three agents. Each agent, given a board state, returns a single legal column.

Mandatory tie-breaking rule (Agents 2–3). Whenever two or more moves are ranked equally best, the agent must choose uniformly at random among them. Without this, deterministic agents replay the identical game every time and the experiment win/draw rates become meaningless. Use a seedable random source and record the seed(s) so your results are reproducible.

Agent 1 — Random Agent [0.5 marks]

Selects a legal move uniformly at random. This is the baseline.

Agent 2 — Rule-Based Agent [1.5 marks]

An agent driven by manually defined, prioritized rules. A reasonable rule set, in priority order:

1. If a move wins immediately, play it.
2. Else, if the opponent has an immediate winning move, block it.
3. Else, prefer central columns.
4. Else, extend your own longest line / create threats.

State your exact rules and their priority order in the report. Apply the tie-breaking rule above when several moves satisfy the same rule.

Agent 3 — Minimax Agent [3 marks]

Implement the Minimax algorithm with:

- Configurable search depth (a parameter).
- Terminal evaluation: win = large positive, loss = large negative, draw = 0 (from the perspective of the agent to move).
- Heuristic evaluation for non-terminal states at the depth limit (see Heuristic guidance below).
- For all reported experiments, use a fixed depth of 4 (you may additionally report an optional depth-6 run for discussion).

Heuristic guidance (Agent 3). A common, defensible evaluation is windowed scoring: slide a length-4 window over every row, column, and diagonal, and score each window by how many of your discs vs. the opponent's it contains (reward 3-in-a-window heavily, 2 modestly; penalise the opponent's the same way), plus a small weight for centre-column control. You may design your own — just describe it precisely in the report.

Requirement 3: Experimental Evaluation [2 marks]

Evaluate your agents head-to-head. At minimum, run these pairings:

Match	Purpose
Random vs Rule-Based	Baseline vs simple heuristics
Rule-Based vs Minimax	Heuristics vs search
Minimax vs Random	Search vs baseline

Protocol:

- Play 30 games per pairing, alternating which agent moves first (~15 each way).
- Run all timing on a single machine and state its specifications (CPU, RAM, OS, language + version); decision times are otherwise not comparable.
- Report the random seed(s) used.

For each pairing, report:

- Win rate (per agent) and draw rate.
- Average decision time per move (per agent).

Present results in tables and discuss what you observe. Note: Connect Four is a solved game (the first player can force a win from the centre), so a sufficiently deep search agent moving first should dominate weaker agents — use this as a sanity check when interpreting your numbers.

Requirement 4: Report [1 mark]

Submit a Report.pdf containing:

1. Introduction
2. System design (engine and architecture; how agents plug in)

3. AI agents (rules, heuristic, search depth, tie-breaking)
4. Experimental results (tables and the required metrics)
5. Discussion — interpret the results and propose other AI techniques that could improve play (e.g. deeper search, better heuristics, MCTS, transposition tables)
6. Team member contributions

Requirement 5: Demonstration Video [0.5 marks]

A 3–5 minute video demonstrating one full run (engine plus at least one AI-vs-AI match), with brief narration of what is happening.

Deliverables

Submit a single ZIP file containing:

1. Source code implemented in python
2. README with a link to the Demonstration video, and instructions to run the program and reproduce the experiments (including seeds)
3. Report.pdf

Academic Integrity

You may use standard libraries for input/output, visualization, and data processing. Any external code, libraries, or online resources must be clearly acknowledged in the report. All group members are expected to contribute, and the contributions section must reflect actual work.